NOTAS DEL CURSO:

PROGRAMACIÓN

PROFESOR:

LUIS ALBERTO MUÑOZ GÓMEZ

Parte I. Programación Estructurada y Procedimental/Modular

Estructura de un programa C/C++

Conceptos de programa y programador

Algoritmo.- Conjunto finito y ordenado de instrucciones libres de ambigüedad y para resolver un problema

Programa.- Implementación de instrucciones basadas en un algoritmo, escritas en un lenguaje de programación y acompañadas de las estructuras de datos necesarias para cumplir con su objetivo

Programador.- Aquel (persona o software) que escribe un programa

La Codificación

Codificación.-Es la concretización de un programa en un código fuente interpretable

- hola mundo, cadena de caracteres ó constante de cadena
- formato del printf
- secuencias de escape: \n, \t, \”, \\

Ciclo de vida del software: requerimientos, análisis, diseño, codificación, pruebas, implantación, explotación

Etapa de implantación: puesta en producción del software obtenido en la etapa de codificación y después de haber sido probado para detección de errores

Etapa de explotación: uso del software

Proceso.- Conjunto de tareas vinculadas ordenadamente que utilizan recursos para generar un producto

El entorno del compilador C/C++

- aplicación de consola
- nombre proyecto
- contenido plantilla
- librería
- compilar
- correr

- código fuente
- código objeto
- qué es un proyecto
- archivos generados
- modos insertar/sobrescribir
- guardar (ctrl+S), compilar (ctrl.+F9), correr (ctrl.+F10), compilar y correr (F9)
- marca (*) de archivo fuente modificado
- evitar los caracteres acentuados, ñ y otros “raros” en los nombres de archivos pues marca error

Variables

Identificador

- nombre único a_1
- sensible al caso
- no empezar con _ por las rutinas de biblioteca
- 31 caracteres significativos

Tipos de datos primitivos

- int entero de tamaño natural en la máquina; %d, %7d
- short int - entero corto
- long int - entero largo
- char - caracter de 1 byte; %c
- float, double; %3.0f, %6.1f, %6f, %.2f
- ANSI
- tamaños
- rangos
- signados y no signados
- enum boolean {NO, YES}

Tipo de dato	Cantidad de bytes	Combinaciones	Rango de Valores
	1	2 a la 8	-128 a 127
short int	2	2 a la 16	-32768 a 32767
Int	4	2 a la 32	-2147483648 a 2147483647
long int	8	2 a la 64	

Declaración y definición de datos Variables

- Variables contra constantes
- Solo declarar
- declarar una o varias
- en minúsculas

Reservación de memoria

- bits y bytes
- reservación de espacio
- imagen de bits en memoria

Inicialización de variables

- definir
- “evitar” declarar y definir al mismo tiempo
- Copiado en celdas de memoria y registros del procesador
- valor izquierdo

Constantes

Constantes const y #define

- hexadecimal
- const
- en mayúsculas
- #define, constantes simbólicas; **#define** *nombre* *texto de reemplazo*
- 12345U (sin signo)
- a=-0.1e-2f
- preprocesamiento para substitución de macros en el texto del programa

Entrada/Salida de datos

Escritura de datos

- printf
- cout

Entrada de datos

- getchar
- scanf
- cin
- valores por defecto a variables

Operadores y Expresiones

Aritméticos: +, -, *, /, división entera, módulo %
Incremento y decremento, antes y después, efectos colaterales: ++, --
Relacionales: >, >=, <, <=, ==, !=
Lógicos: &&, ||, !
De asignación: =, +=, -=, *=, /=
De condición: expr1?expr2:expr3

Prioridad de los operadores (p.58 Kernighan, Ritchie)

Operadores	Asociatividad
`() [] ->`	izquierda a derecha
`! ~ ++ -- + - * & (`*`tipo`*`) sizeof`	derecha a izquierda
`* / %`	izquierda a derecha
`+ -`	izquierda a derecha
`<< >>`	izquierda a derecha
`< <= > >=`	izquierda a derecha
`== !=`	izquierda a derecha
`&`	izquierda a derecha
`^`	izquierda a derecha
`|`	izquierda a derecha
`&&`	izquierda a derecha
`||`	izquierda a derecha
`?:`	derecha a izquierda
`= += -= *= /= %= &= ^= |= <<= >>=`	derecha a izquierda
`,`	izquierda a derecha

Expresiones
- los *operadores* especifican lo que se hará con las variables
- las *expresiones* se forman a partir de operadores y operandos; combinan variables y constantes para producir nuevos valores (generan un *valor izquierdo*)

Proposición: especificación de operaciones a realizar (ejemplo: de cálculo);

- una *proposición* puede ser:
- cualquier expresión,
- una asignación
- una llamada a función
- *proposición nula* “;”

Palabras Reservadas (p. 212 Kernighan, Ritchie)

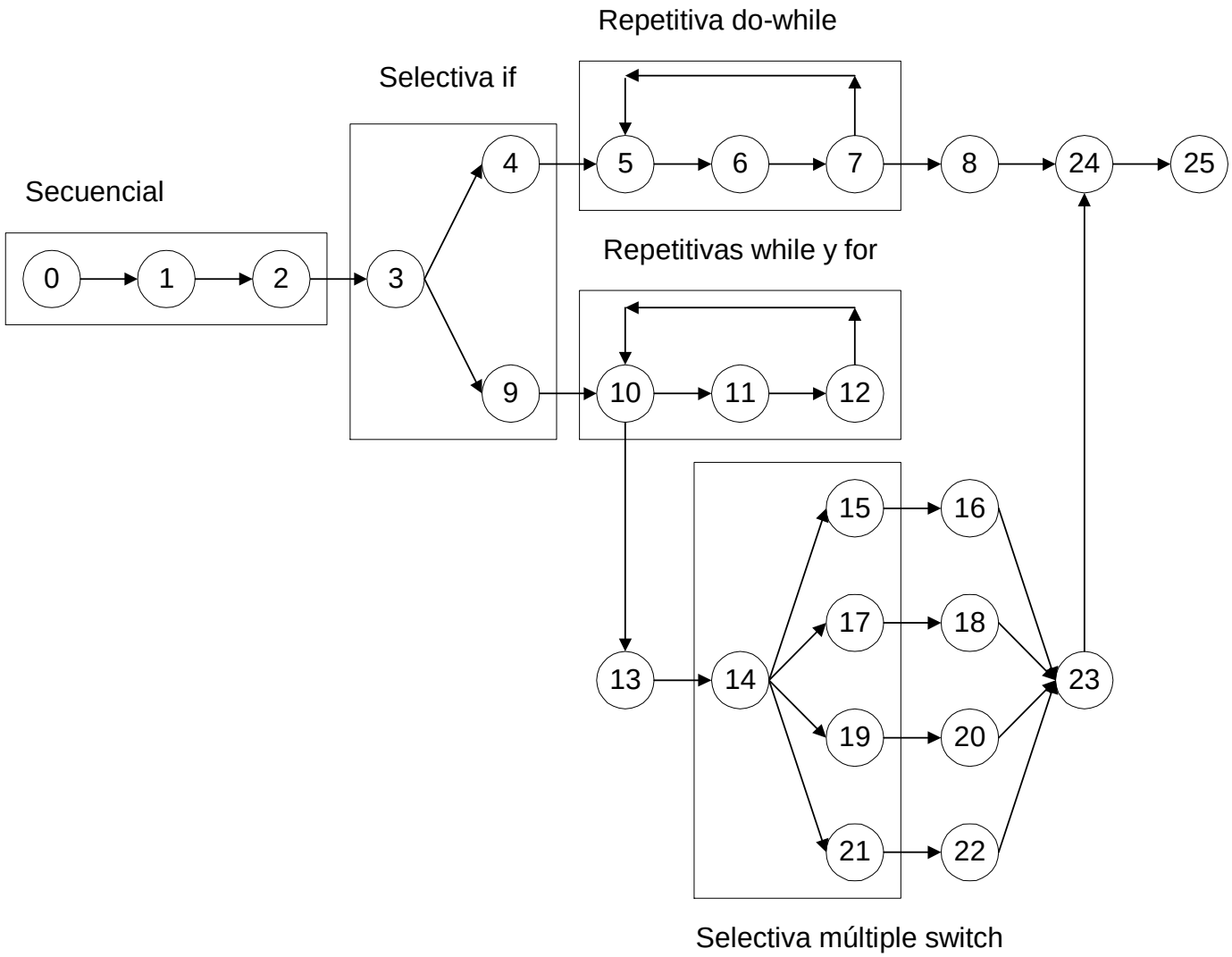
`auto`	`double`	`int`	`struct`
`break`	`else`	`long`	`switch`
`case`	`enum`	`register`	`typedef`
`char`	`extern`	`return`	`union`
`const`	`float`	`short`	`unsigned`
`continue`	`for`	`signed`	`void`
`default`	`goto`	`sizeof`	`volatile`
`do`	`if`	`static`	`while`

Comentarios

- explicación breve de lo que hace un programa
- una línea //
- al final de una línea //
- multilínea /* */
- útil también para depuración de un programa

- conversiones de tipo (casting)

Programación estructurada



Definición de Programación Estructurada

- Estilo de programación que permite la escritura de programas claros, fiables y eficientes mediante 3 estructuras de control de flujo: secuencial, condicional e iterativa y evitando el uso de instrucciones de transferencia incondicional (goto)

El Lenguaje C permite:

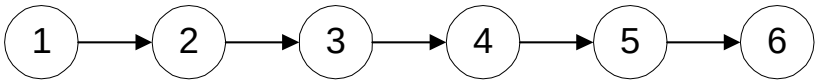
- control de flujo
- agrupación de proposiciones
- toma de decisiones (if-else)
- selección de un caso entre un conjunto de ellos (switch)
- iteración con la condición de paro en la parte superior (while, for) o en la parte inferior (do)

Estrategias de Diseño
Descendente (Top-Down) y Ascendente (Botton-Up)

Ejemplo: Formación Académica

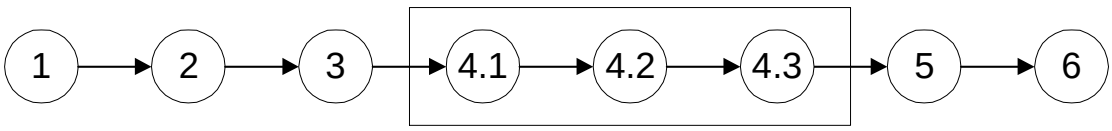
Estructura de control secuencial:

1 Cursar la Primaria
2 Cursar la Secundaria
3 Cursar la Preparatoria
4 Cursar la Licenciatura
5 Cursar la Maestría
6 Cursar el Doctorado



Diseño Descendente...

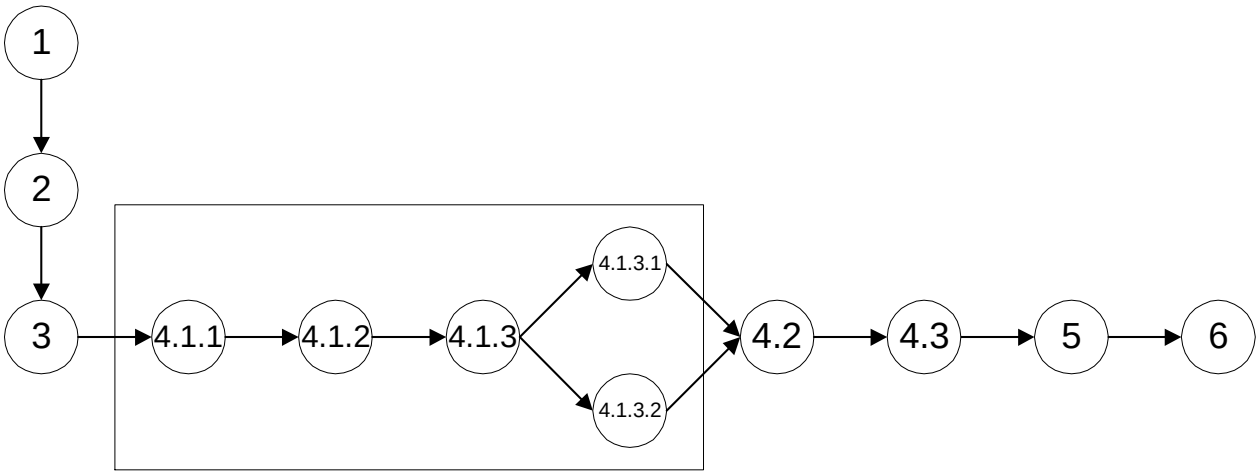
1 Cursar la Primaria
2 Cursar la Secundaria
3 Cursar la Preparatoria
4 Cursar la Licenciatura
  4.1 Cursar la carrera
  4.2 Graduarme de la carrera
  4.3 Titularme de la carrera
5 Cursar la Maestría
6 Cursar el Doctorado



Flujo de Programa (if-else)

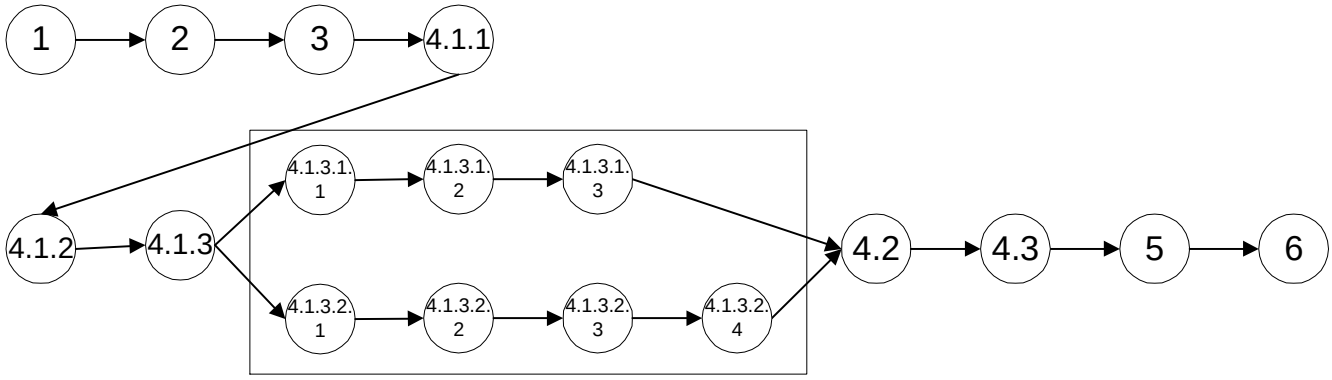
Estructura de control selectiva *if-else*:

1 Cursar la Primaria
2 Cursar la Secundaria
3 Cursar la Preparatoria
4 Cursar la Licenciatura
  4.1 Cursar la carrera
    4.1.1 Seleccionar las 2 carreras que me más me gusten
    4.1.2 Elegir una carrera (Ing. Química ó Ing. en Computación)
    4.1.3 Verificar si estudiar Ingeniería Química *if(expresion){*
      4.1.3.1 Estudiar Ingeniería Química
    }
    else{
      4.1.3.2 Estudiar Ingeniería en Computación
    }
  4.2 Graduarme de la carrera
  4.3 Titularme de la carrera
5 Cursar la Maestría
6 Cursar el Doctorado



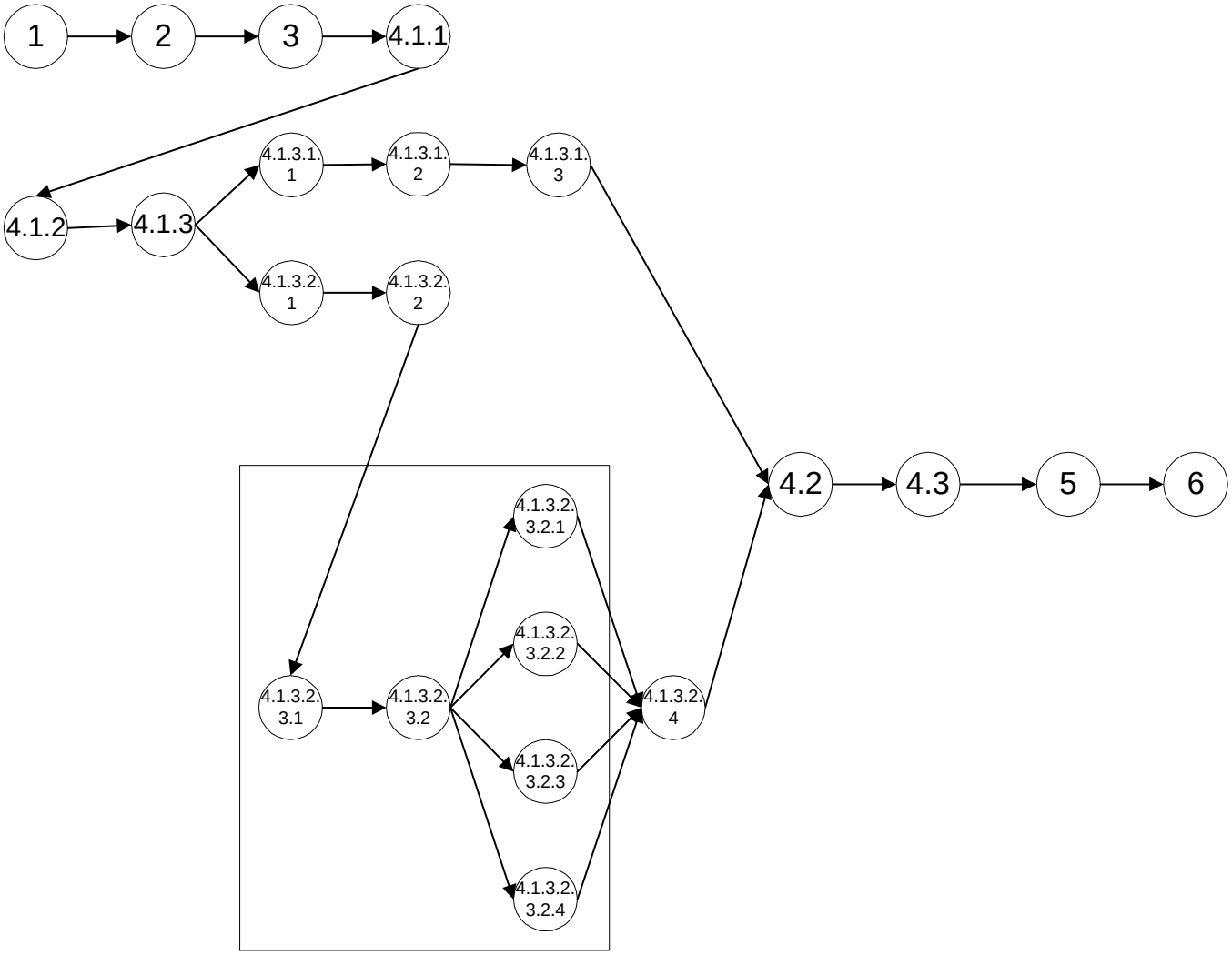
Diseño descendente...

1 Cursar la Primaria
2 Cursar la Secundaria
3 Cursar la Preparatoria
4 Cursar la Licenciatura
  4.1 Cursar la carrera
    4.1.1 Seleccionar las 2 carreras que me más me gusten
    4.1.2 Elegir una carrera (Ing. Química ó Ing. en Computación)
    4.1.3 Verificar si estudiar Ingeniería Química *if(expresion){*
      4.1.3.1 Estudiar Ingeniería Química
        4.1.3.1.1 Cursar las materias más fáciles
        4.1.3.1.2 Cursar la materia más difícil
        4.1.3.1.3 Cursar las materias optativas
    }
    else{
      4.1.3.2 Estudiar Ingeniería en Computación
        4.1.3.2.1 Cursar las materias más fáciles
        4.1.3.2.2 Evaluar un posible descanso de la carrera
        4.1.3.2.3 Cursar la especialidad de la carrera
        4.1.3.2.4 Cursar las materias optativas
    }
  4.2 Graduarme de la carrera
  4.3 Titularme de la carrera
5 Cursar la Maestría
6 Cursar el Doctorado



Estructura de control selectiva múltiple *switch*:

1 Cursar la Primaria
2 Cursar la Secundaria
3 Cursar la Preparatoria
4 Cursar la Licenciatura
  4.1 Cursar la carrera
    4.1.1 Seleccionar las 2 carreras que me más me gusten
    4.1.2 Elegir una carrera (Ing. Química ó Ing. en Computación)
    4.1.3 Verificar si estudiar Ingeniería Química *if(expresion){*
      4.1.3.1 Estudiar Ingeniería Química
        4.1.3.1.1 Cursar las materias más fáciles
        4.1.3.1.2 Cursar la materia más difícil
        4.1.3.1.3 Cursar las materias optativas
    }
    else{
      4.1.3.2 Estudiar Ingeniería en Computación
        4.1.3.2.1 Cursar las materias más fáciles
        4.1.3.2.2 Evaluar un posible descanso de la carrera
        4.1.3.2.3 Cursar la especialidad de la carrera
          4.1.3.2.3.1 Elegir una de las especialidades
          4.1.3.2.3.2 Verificar por la opción a seguir *switch(var){*
            4.1.3.2.3.2.1 Cursar por Software de Sistemas
            4.1.3.2.3.2.2 Cursar por Sistemas Computacionales
            4.1.3.2.3.2.3 Cursar por Sistemas Digitales
            4.1.3.2.3.2.4 Cursar por Sistemas de Información
          }
        4.1.3.2.4 Cursar las materias optativas
    }
  4.2 Graduarme de la carrera
  4.3 Titularme de la carrera
5 Cursar la Maestría
6 Cursar el Doctorado


1
2
3
4.1.1
4.1.2
4.1.3
4.1.3.1.1
4.1.3.1.2
4.1.3.1.3
4.1.3.2.1
4.1.3.2.2
4.1.3.2.3.1
4.1.3.2.3.2
4.1.3.2.3.2.1
4.1.3.2.3.2.2
4.1.3.2.3.2.3
4.1.3.2.3.2.4
4.1.3.2.4
4.2
4.3
5
6

```
anio<-2000
anio/400
multiplo de 400 --- (anio%400)==0

si (es un año múltiplo de 400) entonces{
  es un año bisiesto
}
sino si ((es un año múltiplo de 4) y
         (no es un año múltiplo de 100)) entonces{
  es un año bisiesto
}
sino{
  no es un año bisiesto
}

si ((es un año múltiplo de 400) ó
    ((es un año múltiplo de 4) y (no es un año múltiplo de
100))){
  es un año bisiesto
}
sino{
  no es un año bisiesto
}
ó es ||
y es &&
```

Notas de clase

multi4 = ”el año es un múltiplo de 4” // =(anio%4)==0
multi100 = ”el año es un múltiplo de 100”
multi400 = ”el año es un múltiplo de 400”
esFebrero = mes==2
diaEntreUnoVeintiocho = (1<=dia) && (dia<=28)
dia29 = dia==29

((multi4 Λ ¬multi100) V multi400) => anioBisiesto

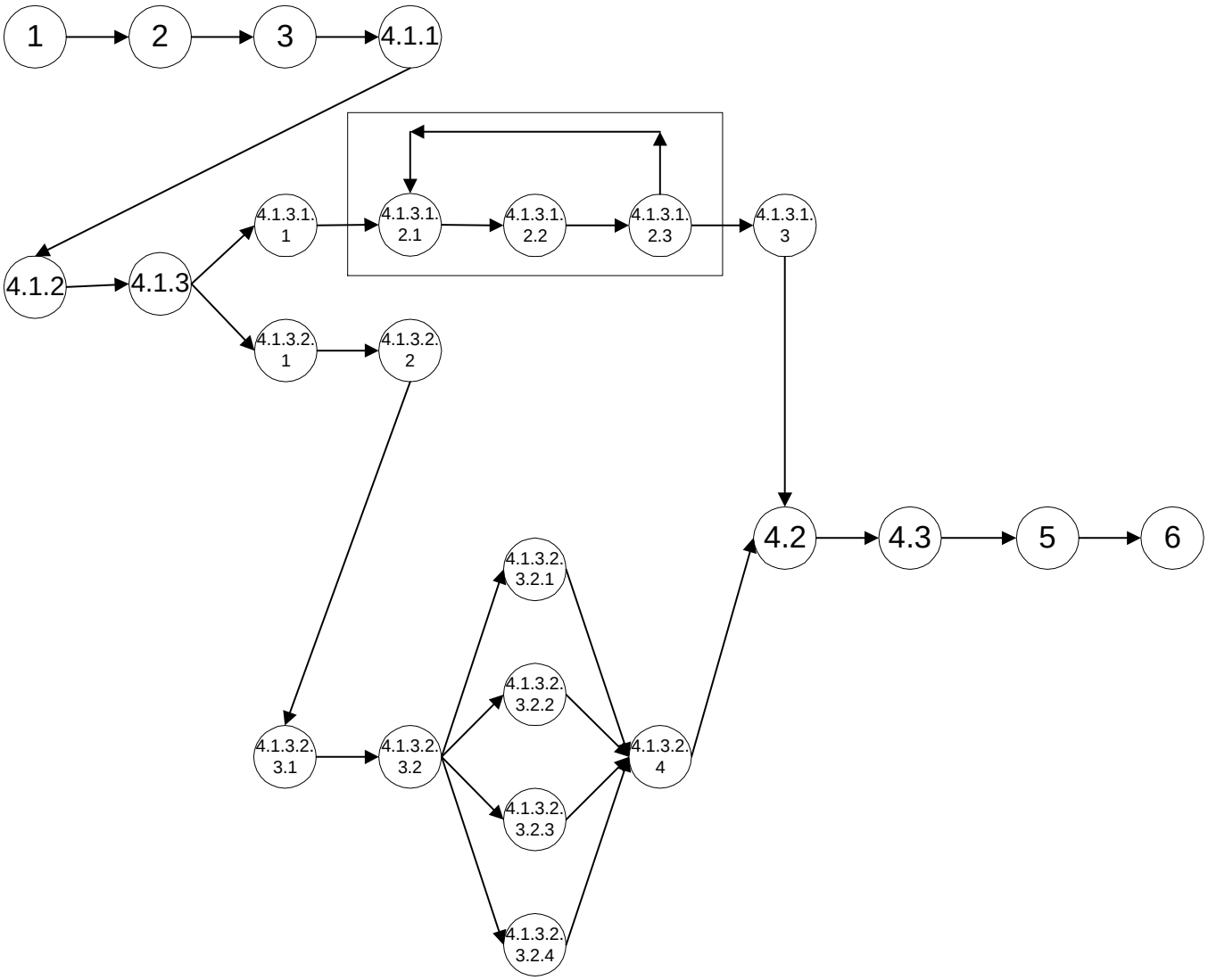
(esFebrero Λ (diaEntreUnoVeintiocho V (dia29 Λ anioBisiesto))) => fechaValida

Λ en C es &&
V en C es ||
=> en C es un if (...){…}

Flujo de Programa (do-while, while y for)

Estructura de control iterativa con condición de paro en la parte inferior (verificación al final) *do-while*:

1 Cursar la Primaria
2 Cursar la Secundaria
3 Cursar la Preparatoria
4 Cursar la Licenciatura
  4.1 Cursar la carrera
    4.1.1 Seleccionar las 2 carreras que me más me gusten
    4.1.2 Elegir una carrera (Ing. Química ó Ing. en Computación)
    4.1.3 Verificar si estudiar Ingeniería Química *if(expresion){*
      4.1.3.1 Estudiar Ingeniería Química
        4.1.3.1.1 Cursar las materias más fáciles
        4.1.3.1.2 Cursar la materia más difícil
          4.1.3.1.2.1 Iniciar *do{*
          4.1.3.1.2.2 Evaluar mi desempeño en el curso
          4.1.3.1.2.3 Verificar si cursar de nuevo *}while(expr)*
        4.1.3.1.3 Cursar las materias optativas
    }
    else{
      4.1.3.2 Estudiar Ingeniería en Computación
        4.1.3.2.1 Cursar las materias más fáciles
        4.1.3.2.2 Evaluar un posible descanso de la carrera
        4.1.3.2.3 Cursar la especialidad de la carrera
          4.1.3.2.3.1 Elegir una de las especialidades
          4.1.3.2.3.2 Verificar por la opción a seguir *switch(var){*
            4.1.3.2.3.2.1 Cursar por Software de Sistemas
            4.1.3.2.3.2.2 Cursar por Sistemas Computacionales
            4.1.3.2.3.2.3 Cursar por Sistemas Digitales
            4.1.3.2.3.2.4 Cursar por Sistemas de Información
          }
        4.1.3.2.4 Cursar las materias optativas
    }
  4.2 Graduarme de la carrera
  4.3 Titularme de la carrera
5 Cursar la Maestría
6 Cursar el Doctorado



Contadores, Acumuladores y Banderas

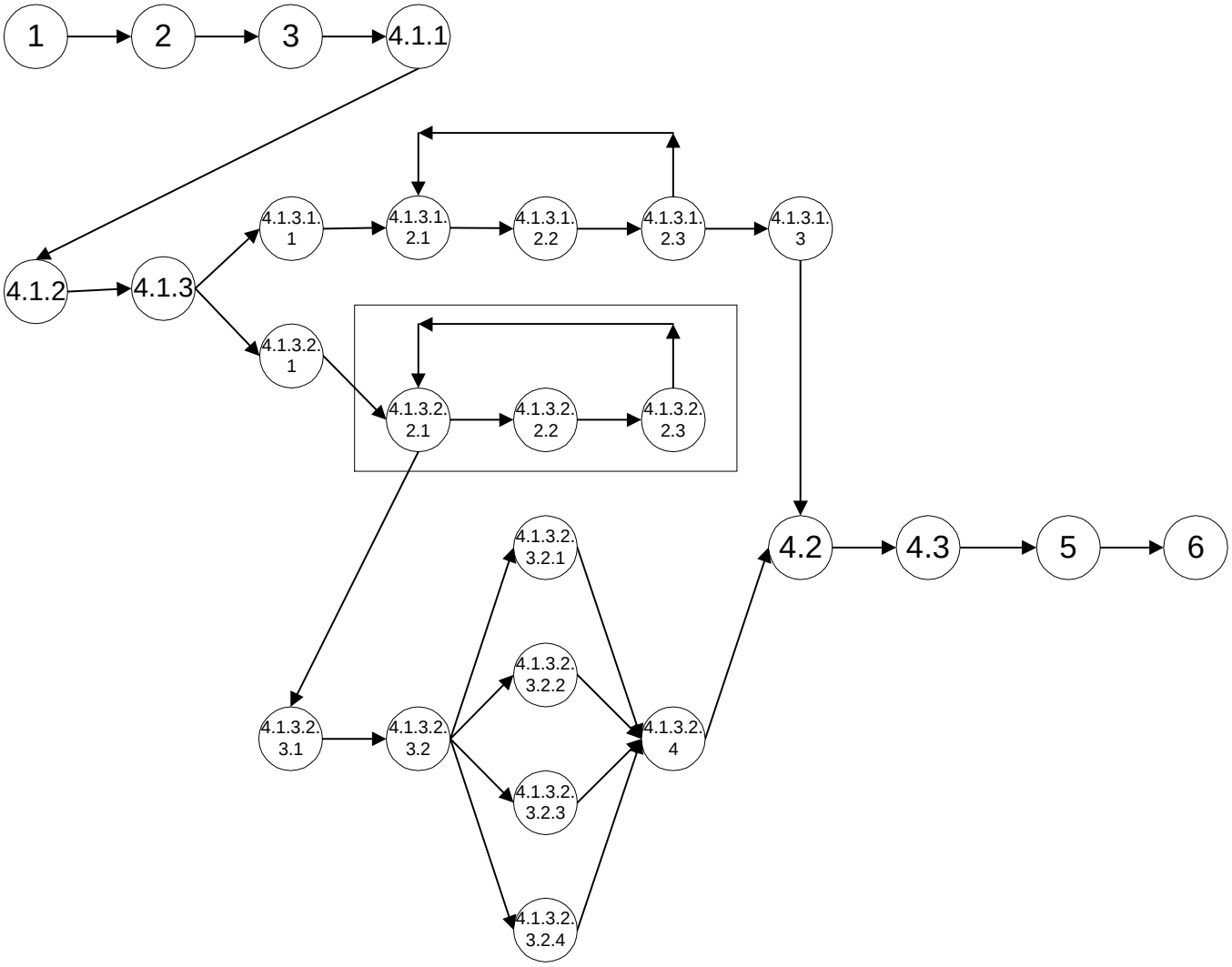
Contador.- Variable entera cuya función es contar de 1 en 1 ya sea de forma incremental ó decremental; generalmente se usa al interior de una estructura de control iterativa

Acumulador.-Variable de tipo entero ó flotante que sirve para almacenar de forma incremental distintos valores

Bandera.-variable (en C es de tipo entero) que puede asumir un valor de verdad (verdadero ó falso) y que representa ya sea la realización de un bloque de instrucciones ó bien la continuidad en la realización del mismo, según sea la estructura de control a la que se le asocie

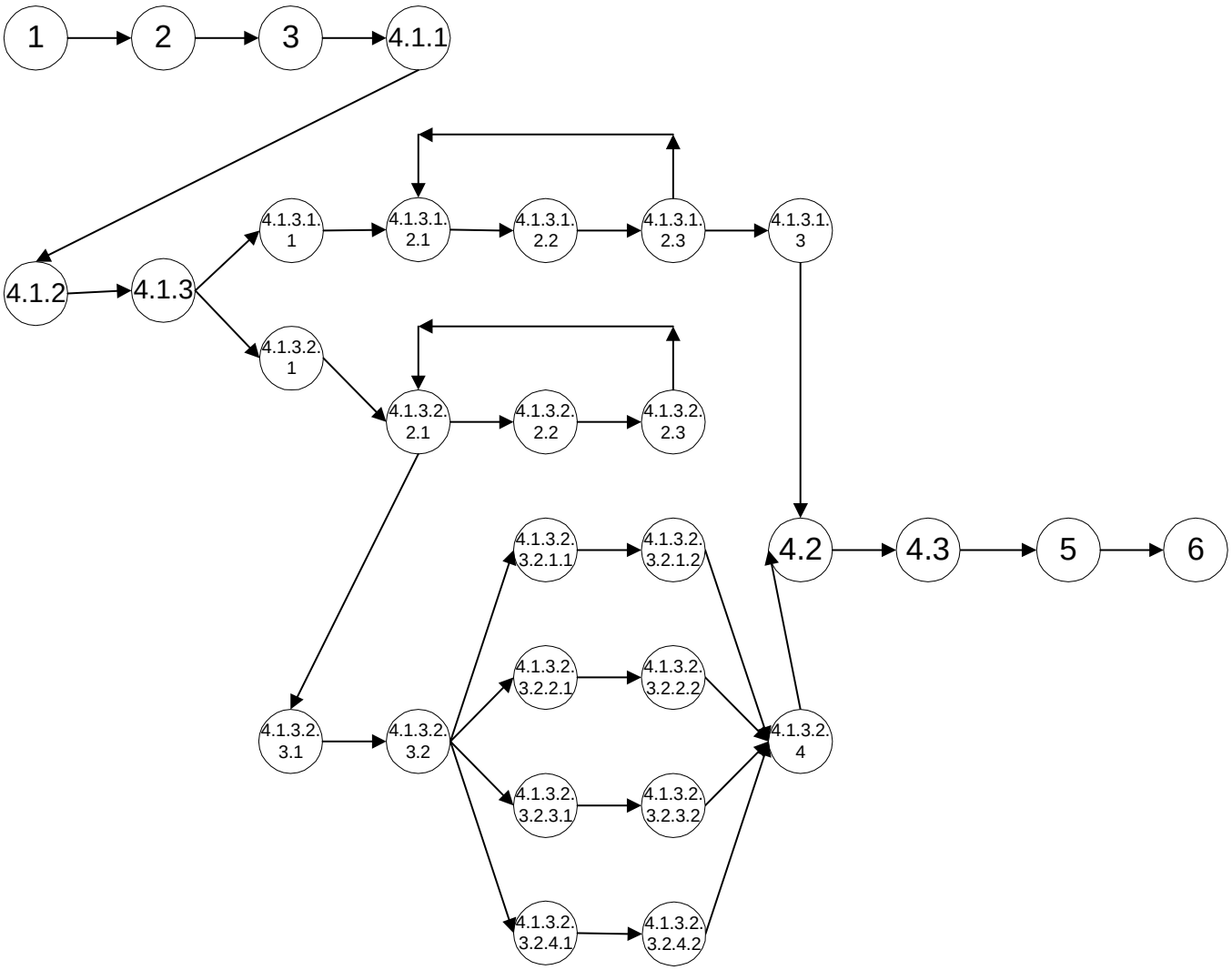
Estructura de control iterativa con condición de paro en la parte superior (verificación al inicio) *while ó for*:

1 Cursar la Primaria
2 Cursar la Secundaria
3 Cursar la Preparatoria
4 Cursar la Licenciatura
  4.1 Cursar la carrera
    4.1.1 Seleccionar las 2 carreras que me más me gusten
    4.1.2 Elegir una carrera (Ing. Química ó Ing. en Computación)
    4.1.3 Verificar si estudiar Ingeniería Química *if(expresion){*
      4.1.3.1 Estudiar Ingeniería Química
        4.1.3.1.1 Cursar las materias más fáciles
        4.1.3.1.2 Cursar la materia más difícil
          4.1.3.1.2.1 Iniciar *do{*
          4.1.3.1.2.2 Evaluar mi desempeño en el curso
          4.1.3.1.2.3 Verificar si cursar de nuevo *}while(expr)*
        4.1.3.1.3 Cursar las materias optativas
    }
    else{
      4.1.3.2 Estudiar Ingeniería en Computación
        4.1.3.2.1 Cursar las materias más fáciles
        4.1.3.2.2 Evaluar un posible descanso de la carrera
          4.1.3.2.2.1 Verificar si falto un semestre *while(expr){*
          4.1.3.2.2.2 Trabajar un tiempo
          4.1.3.2.2.3 Regularizarme en conocimientos necesarios
          }
        4.1.3.2.3 Cursar la especialidad de la carrera
          4.1.3.2.3.1 Elegir una de las especialidades
          4.1.3.2.3.2 Verificar por la opción a seguir *switch(var){*
            4.1.3.2.3.2.1 Cursar por Software de Sistemas
            4.1.3.2.3.2.2 Cursar por Sistemas Computacionales
            4.1.3.2.3.2.3 Cursar por Sistemas Digitales
            4.1.3.2.3.2.4 Cursar por Sistemas de Información
          }
        4.1.3.2.4 Cursar las materias optativas
    }
  4.2 Graduarme de la carrera
  4.3 Titularme de la carrera
5 Cursar la Maestría
6 Cursar el Doctorado



...y el programa completo quedaría con algo más de diseño descendente al interior del switch...

1 Cursar la Primaria
2 Cursar la Secundaria
3 Cursar la Preparatoria
4 Cursar la Licenciatura
  4.1 Cursar la carrera
    4.1.1 Seleccionar las 2 carreras que me más me gusten
    4.1.2 Elegir una carrera (Ing. Química ó Ing. en Computación)
    4.1.3 Verificar si estudiar Ingeniería Química *if(expresion){*
      4.1.3.1 Estudiar Ingeniería Química
        4.1.3.1.1 Cursar las materias más fáciles
        4.1.3.1.2 Cursar la materia más difícil
          4.1.3.1.2.1 Iniciar *do{*
          4.1.3.1.2.2 Evaluar mi desempeño en el curso
          4.1.3.1.2.3 Verificar si cursar de nuevo *}while(expr)*
        4.1.3.1.3 Cursar las materias optativas
    }
    else{
      4.1.3.2 Estudiar Ingeniería en Computación
        4.1.3.2.1 Cursar las materias más fáciles
        4.1.3.2.2 Evaluar un posible descanso de la carrera
          4.1.3.2.2.1 Verificar si falto un semestre *while(expr){*
          4.1.3.2.2.2 Trabajar un tiempo
          4.1.3.2.2.3 Regularizarme en conocimientos necesarios
          }
        4.1.3.2.3 Cursar la especialidad de la carrera
          4.1.3.2.3.1 Elegir una de las especialidades
          4.1.3.2.3.2 Verificar por la opción a seguir *switch(var){*
            4.1.3.2.3.2.1 Cursar por Software de Sistemas
              4.1.3.2.3.2.1.1 Cursar las materias más fáciles
              4.1.3.2.3.2.1.2 Cursar la materia más difícil
            4.1.3.2.3.2.2 Cursar por Sistemas Computacionales
              4.1.3.2.3.2.2.1 Cursar las materias más fáciles
              4.1.3.2.3.2.2.2 Cursar la materia más difícil
            4.1.3.2.3.2.3 Cursar por Sistemas Digitales
              4.1.3.2.3.2.3.1 Cursar las materias más fáciles
              4.1.3.2.3.2.3.2 Cursar la materia más difícil
            4.1.3.2.3.2.4 Cursar por Sistemas de Información
              4.1.3.2.3.2.4.1 Cursar las materias más fáciles
              4.1.3.2.3.2.4.2 Cursar la materia más difícil
          }
        4.1.3.2.4 Cursar las materias optativas
    }
  4.2 Graduarme de la carrera
  4.3 Titularme de la carrera
5 Cursar la Maestría
6 Cursar el Doctorado


1
2
3
4.1.1
4.1.2
4.1.3
4.1.3.1.1
4.1.3.1.2.1
4.1.3.1.2.2
4.1.3.1.2.3
4.1.3.1.3
4.1.3.2.1
4.1.3.2.2.1
4.1.3.2.2.2
4.1.3.2.2.3
4.1.3.2.3.1
4.1.3.2.3.2
4.1.3.2.3.2.1.1
4.1.3.2.3.2.1.2
4.1.3.2.3.2.2.1
4.1.3.2.3.2.2.2
4.1.3.2.3.2.3.1
4.1.3.2.3.2.3.2
4.1.3.2.3.2.4.1
4.1.3.2.3.2.4.2
4.1.3.2.4
4.2
4.3
5
6

Arreglos

Definiciones

Estructura de Datos.- Colección de datos organizados de un modo particular; pueden ser de dos tipos:

a) estáticas: son aquellas en las que se asigna una cantidad fija de memoria cuando se declara la variable
b) dinámicas: son aquellas cuya ocupación de memoria puede aumentar y disminuir en tiempo de ejecución a medida que el programa progresa

Arreglo.-Estructura de datos en la que se almacena una colección de datos del mismo tipo; ejemplo: los salarios de los empleados de una empresa. Es una lista de un número finito de n elementos del mismo tipo que se caracteriza por:

1) almacenar los elementos del array en posiciones de memoria continua
2) tener un único nombre de variable; ejemplo: salarios

Vectores

- Arreglos de una dimensión

Matrices

- Arreglos de dos dimensiones

Programación Procedimental/Modular: Funciones

- Módulos de programa
- Pueden regresar valores de tipos básicos, estructuras, uniones o apuntadores
- Las *funciones* contienen proposiciones encerradas todas entre llaves

- Funciones sin parámetros
- Prototipos de funciones
- Funciones con parámetros por valor
- Funciones predefinidas (matemáticas y de cadena)

Variables

- Variables locales automáticas
- Declaradas en modalidad estructurada por bloques
- Pueden ser internas a una función o visibles al programa completo

Recursividad

Es la forma en la cual se especifica un proceso basado en su propia definición.

Factorial:

$$f(n)=\begin{cases} n=0: 1 \\ n>0: n*f(n-1) \\ \text{otro: indefinido} \end{cases}$$

Sucesión de Fibonacci:

$$f_0 = 0$$

$$f_1 = 1$$

$$f_n = f_{n-1} + f_{n-2} \text{ para } n = 2, 3, 4, 5, \ldots$$

Apuntadores

Apuntador.-Variable que contiene la dirección de memoria donde se almacena alguna información

Estructuras

Tipo de Dato Estructurado.-Colección de 2 o más tipos de datos primitivos que generan un nuevo tipo de abstracción: Persona(código, nombre, rfc, salario)

Parte II. Programación Orientada a Objetos

Strings

string.-Tipo de dato que representa a una secuencia de caracteres, también utilizado como "constante de cadena".

Introducción a la Programación Orientada a Objetos

Programación Procedimental contra la Orientada a Objetos

Programación Procedimental

El paradigma de programación original (y probablemente aun el más comúnmente usado) es:

- Decidir cuales procedimientos quieres; usar los mejores algoritmos que puedas encontrar

El foco está en el procesamiento, el algoritmo necesario para ejecutar la computación deseada.

Programación Orientada a Objetos:

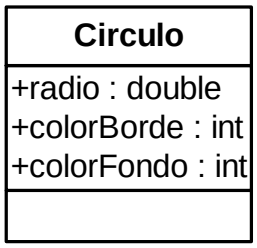
El paradigma de programación es:

- Decidir cuales clases deseas; proveer un conjunto completo de operaciones para cada clase; vuelve explícito lo común mediante el uso de la herencia

Introducción al Pseudocódigo

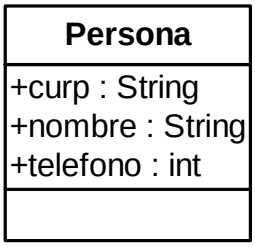
Abstracción

```
Circulo{
  radio:real
  colorBorde:entero
  colorFondo:entero
}
```




```
class Circulo{ //como registro
    public:
        double radio;
        int colorBorde;
        int colorFondo;
};

int main()
{
    Circulo c;
    c.radio=10;
    c.colorBorde=11;
    c.colorFondo=14;
    cout << "radio es " << c.radio << endl;
    cout << "colorBorde es " << c.colorBorde << endl;
    cout << "colorFondo es " << c.colorFondo << endl;
    return 0;
}
```




```
class Persona{ //como registro
    public:
        string curp;
        string nombre;
        int telefono;
};

int main()
{
    Persona p;
    p.curp="MUGL790912IC7";
    p.nombre="Luis Alberto";
    p.telefono=1234567;
    cout << "curp es " << p.curp << endl;
    cout << "nombre es " << p.nombre << endl;
    cout << "telefono es " << p.telefono << endl;
    return 0;
}
```

Objetos y Clases

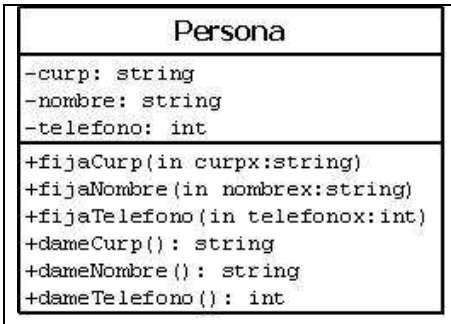
```
Circulo{
  radio:real
  colorBorde:entero
  colorFondo:entero
  PI:constante<-3.1416

  fijaRadio(radiox){
    radio<-radiox
  }
  fijaColorBorde(colorBordex){
    colorBorde<-colorBordex
  }
  fijaColorFondo(colorFondox){
    colorFondo<-colorFondox
  }
  <-dameRadio(){
    regresa radio
  }
  <-dameColorBorde(){
    regresa colorBorde
  }
  <-dameColorFondo(){
    regresa colorFondo
  }
  <-dameArea(){
    regresa PI*radio*radio
  }
}
```

Circulo
-radio: float -colorBorde: int -colorFondo: int
+fijaRadio(in radiox:float) +fijaColorBorde(in colorBordex:int) +fijaColorFondo(in colorFondox:int) +dameRadio(): float +dameColorBorde(): int +dameColorFondo(): int +dameArea(): float

```
class Circulo{
    static const float PI=3.1416;
    double radio;
    int colorBorde;
    int colorFondo;
public:
    void fijaRadio(double radiox){
        radio=radiox;
    }
    void fijaColorBorde(int colorBordex){
        colorBorde=colorBordex;
    }
    void fijaColorFondo(int colorFondox){
        colorFondo=colorFondox;
    }
    double dameRadio(){
        return radio;
    }
    int dameColorBorde(){
        return colorBorde;
    }
    int dameColorFondo(){
        return colorFondo;
    }
    float dameArea(){
        return PI*radio*radio;
    }
};

int main()
{
    Circulo c;
    c.fijaRadio(10);
    c.fijaColorBorde(11);
    c.fijaColorFondo(14);
    cout << "radio es " << c.dameRadio() << endl;
    cout << "colorBorde es " << c.dameColorBorde() << endl;
    cout << "colorFondo es " << c.dameColorFondo() << endl;
    cout << "area es " << c.dameArea() << endl;
    return 0;
}
```




```
class Persona{ //como clase
    string curp;
    string nombre;
    int telefono;
public:
    void fijaCurp(string curpx){
        curp=curpx;
    }
    void fijaNombre(string nombrex){
        nombre=nombrex;
    }
    void fijaTelefono(int telefonox){
        telefono=telefonox;
    }
    string dameCurp(){
        return curp;
    }
    string dameNombre(){
        return nombre;
    }
    int dameTelefono(){
        return telefono;
    }
};

int main()
{
    Persona p;
    p.fijaCurp("MUGL790912IC7");
    p.fijaNombre("Luis Alberto");
    p.fijaTelefono(1234567);
    cout << "curp es " << p.dameCurp() << endl;
    cout << "nombre es " << p.dameNombre() << endl;
    cout << "telefono es " << p.dameTelefono() << endl;
    return 0;
}
```

Ocultamiento de Información

El *ocultamiento de información* permite evitar que el código de una *aplicación cliente* tenga acceso a los datos de una *clase* que bien pueden requerir un trato cuidadoso, así cualquier solicitud para modificar o consultar el estado de un *objeto* deba ser mediante el uso de *métodos de interfaz* (públicos) y bajo las restricciones que en ellos se especifiquen.

Instanciación

Creación, inicialización y limpiado de objetos

Véase ejemplos:
“Ejemplos PRO\Orientada a Objetos\03 Constructores e inicializacion”

Fortalezas y debilidades de la POO

Fortalezas:

- Modelado en la dimensión estructural de los objetos
- Modelado semántico entidad-relacionamiento
- Identificación y especificación de objetos, clases, métodos, atributos, asociaciones dependientes de dominio
- Jerarquías de herencia para reutilización de código

Debilidades:

- Criterios consistentes de particionamiento de la complejidad. Deben existir criterios objetivos y prácticos para agregar clases o particionar sistemas
- Reutilización de la especificación. La máxima reutilización ocurre al interior de dominios específicos de aplicación.
- Modelado funcional vs end-to-end. Por la descomposición funcional top-down, las funciones se deben subordinar a los objetos.
- Validación del usuario en el modelado del dominio del problema
- Estimación o dimensionamiento de los sistemas. Está bajo investigación.

Introducción al modelado Orientado a Objetos y el Lenguaje Unificado de Modelado (UML)

UML es un lenguaje de modelado visual que se usa para especificar, visualizar, construir y documentar artefactos de un sistema de software. Se usa para entender, diseñar, configurar, mantener y controlar la información sobre los sistemas a construir.

Breve historia de UML (Unified Modelling Language)

UML nació como una notación estándar de la construcción de modelos. UML comenzó a gestarse en Octubre de 1994. UML es el resultado de la fusión de dos métodos, el OMT de Rumbaugh y el método de Booch para diseño de software orientado a objetos. El primer borrador nació en Octubre de 1995.

Quién usa UML?

Problemas complejos requieren particularmente suficiente planeación para evitar problemas durante el desarrollo y más en las etapas de mantenimiento y actualización donde los cambios son más arduos y costosos. UML permite a personas de diferentes disciplinas, trabajar juntos en identificar y resolver problemas antes de que ocurran.

Los desarrolladores y analistas de negocios pueden mapear los requerimientos en un lenguaje. No hay necesidad de que consideren la tecnología subyacente en esta etapa, que permite a no-programadores estar más envueltos en el proceso de desarrollo.

UML es utilizado en sistemas complejos para capturar no solo la información del software orientado a objetos, sino también las reglas de negocio. UML es una elección natural como lenguaje independiente de plataforma.

Muchos desarrolladores solo usan la aproximación de “solo código”. Esto causa severos problemas cuando el sistema se expande o los desarrolladores originales ya no trabajan más en el proyecto.

Ventajas

1. Es un lenguaje formal, cada elemento tiene un significado bien definido de modo que no se presta a malentendidos.
2. Es conciso, dado que el lenguaje tiene una notación, simple y sencilla
3. Es comprensible, por describir lo más importante del sistema
4. Es escalable, por ser suficientemente formal para manejar proyectos de modelado masivo de sistemas masivos así como pequeños proyectos, evitando excesos
5. Se construyó sobre lecciones aprendidas, por ser la culminación de las mejores prácticas de la comunidad orientada a objetos
6. Es el estándar, dado que UML es controlado por un grupo de manejadores de estándares con contribuciones activas de un grupo mundial de proveedores y

académicos. El estándar asegura la habilidad de transformación de UML y su interoperabilidad, a modo de no estar casado con un producto particular.

Objetivos

- Ofrecer a programadores un lenguaje de modelado visual listo para usarse que ayude en el desarrollo e intercambio de modelos significativos
- Independencia de lenguajes de programación y procesos de desarrollo
- Proporcionar mecanismos de extensión y especialización
- Proporcionar una base formal para entender el lenguaje de modelado
- Fomentar el crecimiento en el mercado las herramientas para orientado a objetos
- Soportar conceptos de desarrollo de alto nivel como pueden ser colaboraciones, frameworks, patterns y componentes
- Integrar las mejores prácticas utilizadas hasta el momento

Apuntador this →

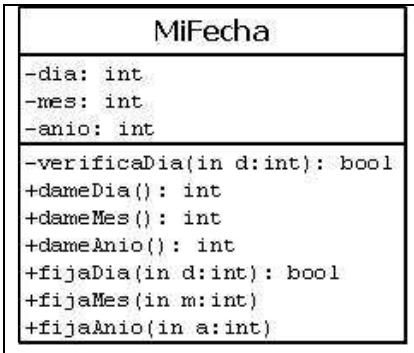
Véase ejemplo: “Ejemplos PRO\Orientada a Objetos\05 this”

Encapsulamiento

El *encapsulamiento* permite que ocultemos los detalles de implementación de una clase y que si necesitamos mejorar su funcionamiento, siempre y cuando se mantengan sin cambios sus *métodos de interfaz*, las aplicaciones cliente continuarán funcionando correctamente.

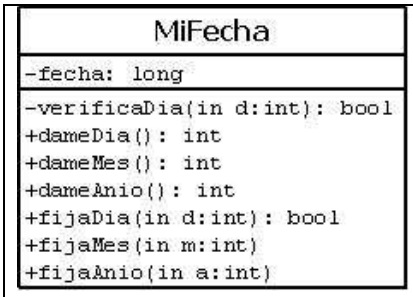
```
class MiFecha{  //Fecha como registro
public:
    int dia;
    int mes;
    int anio;
}

//el problema: libre acceso del exterior
int main() {
    MiFecha f;
    //asignarle mes y año
    f.dia=31;  //es posible, pero de qué mes?
    cout << "dia=" << f.dia;
}
```




```
class MiFecha{  //Fecha como clase
    int dia;
    int mes;
    int anio;
    bool verificaDia(int d);
public:
    bool fijaDia(int d){
        dia=d;  //requiere validar antes
        return true;  //no siempre es válido
    }
    int dameDia(){
        return dia;
    }
    //completar otros dame() y fija()
}

//la solución
int main() {
    MiFecha f;
    //f.dia=31;  //no compila por ocultamiento
    f.fijaDia(31);
    cout << "dia=" << f.dameDia();
    return 0;
}
```

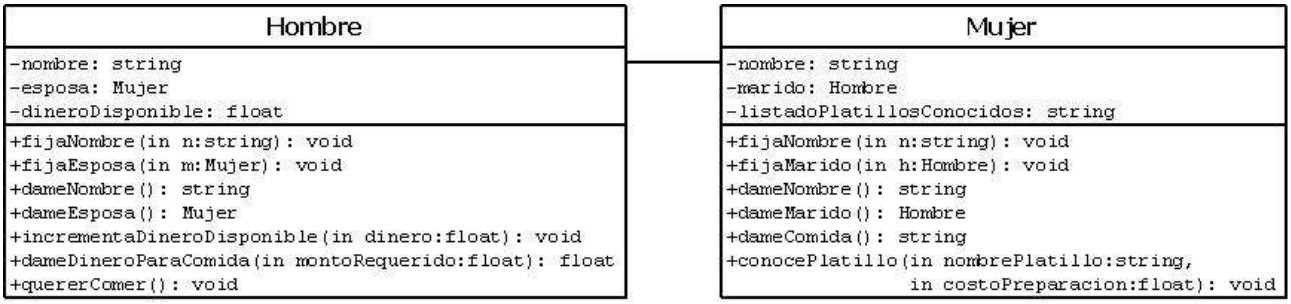



```
class MiFecha{
    //fecha en milisegundos a 64 bits
    long fecha;
    bool verificaDia(int d);
public:
    bool fijaDia(int d){
        //validar d
        if (diaxValido){
            //calcular fechax nueva
            fecha=fechax;
        }
        return diaxValido;
    }
    int dameDia(){
        //calcular a partir fecha en el pasado
        return dia;
    }
    //...
}
int main() {
    MiFecha f;
    //invocar a fijaMes() y fijaAnio()
    f.fijaDia(31);
    cout << "dia=" << f.dameDia();
    return 0;
}
```

Comunicación entre objetos

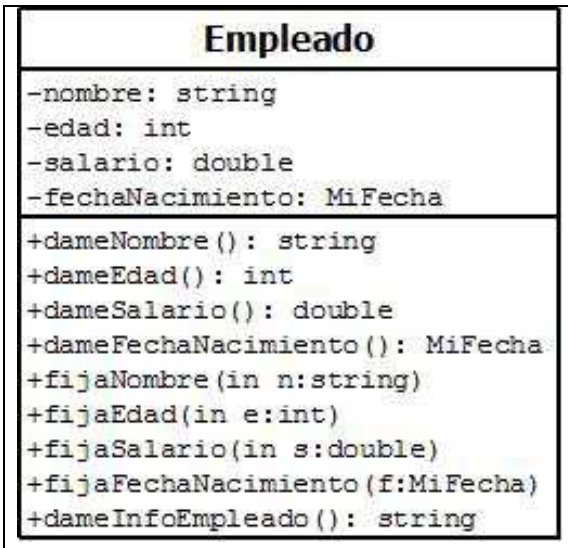
Véase ejemplo: “Ejemplos PRO\Orientada a Objetos\07 Comunicacion”

Comunicación entre dos objetos



Herencia

Abstracción base:

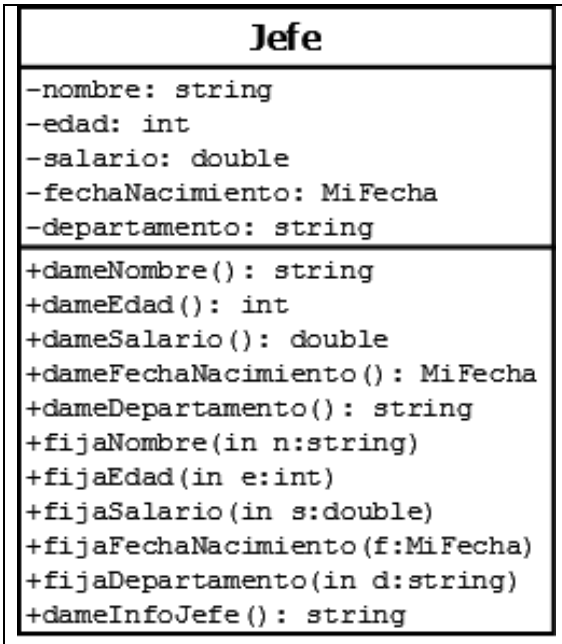



```
class MiFecha{
public:
    int anio;
    int mes;
    int dia;
};

class Empleado{
    string nombre;
    int edad;
    double salario;
    MiFecha fechaNacimiento;
public:
    Empleado(string nombre, int edad,double salario){
        this->nombre = nombre;
        this->edad = edad;
        this->salario=salario;
        fechaNacimiento.anio=1979;
        fechaNacimiento.mes=9;
        fechaNacimiento.dia=12;
    }
    string dameInfoEmpleado(){
        stringstream info;
        info << "Nombre:" << nombre << "\nEdad:" << edad <<
            "\nSalario:" << salario <<
            "\nFechaNacimiento:" << fechaNacimiento.anio << "/"
<<
            fechaNacimiento.mes << "/" << fechaNacimiento.dia <<
"\n";
        return info.str();
    }
};
int main()
{
    Empleado luis("Luis",31,18000);
    cout << "---\n" << luis.dameInfoEmpleado();
    return 0;
}
```

Véase ejemplo: “Ejemplos PRO\Orientada a Objetos\08 Herencia\01 Abstraccion base”

Problema:



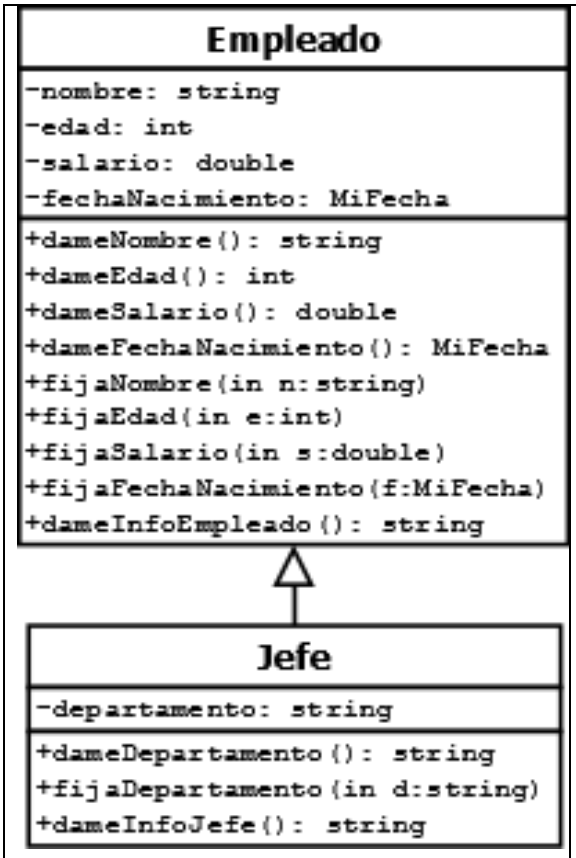

```
class MiFecha{
public:
   int anio;
   int mes;
   int dia;
};

class Jefe{
   string nombre;
   int edad;
   double salario;
   MiFecha fechaNacimiento;
   string departamento;
public:
   Jefe(string nombre, int edad,double salario,string
departamento){
      this->nombre = nombre;
      this->edad = edad;
      this->salario=salario;
      fechaNacimiento.anio=1979;
      fechaNacimiento.mes=9;
      fechaNacimiento.dia=12;
      this->departamento=departamento;
   }
   string dameInfoJefe(){
      stringstream info;
      info << "Nombre:" << nombre << "\nEdad:" << edad <<
         "\nSalario:" << salario <<
         "\nFechaNacimiento:" << fechaNacimiento.anio << "/"
<<
         fechaNacimiento.mes << "/" << fechaNacimiento.dia <<
"\n";
      return info.str()+"Departamento:"+departamento+"\n";
   }
};

int main()
{
   Jefe luisAlberto("Luis Alberto",32,30000,"Computacion");
   cout << "---\n" << luisAlberto.dameInfoJefe();
   return 0;
}
```

Véase ejemplo: “Ejemplos PRO\Orientada a Objetos\08 Herencia\02 Problema”

Solución:



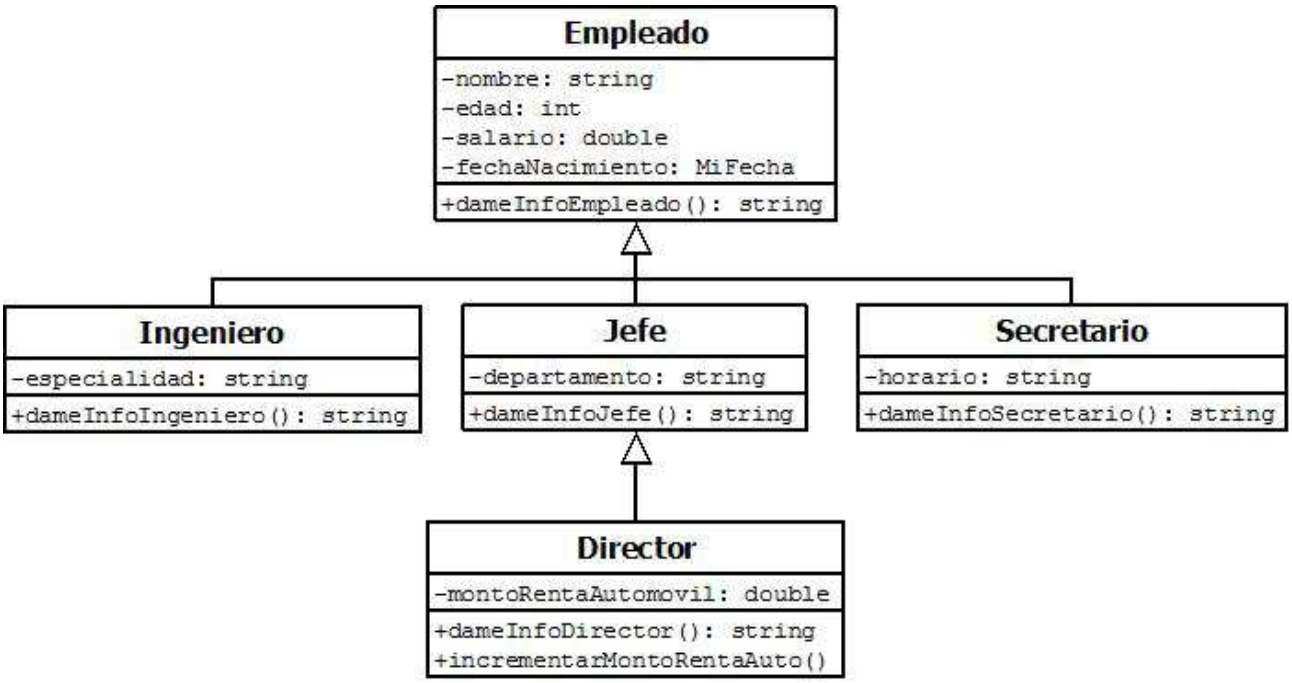

```
class Jefe : public Empleado{
   string departamento;
public:
   Jefe(string nombre, int edad,double salario,string
departamento):
   Empleado(nombre, edad, salario){
      this->departamento=departamento;
   }
   string dameInfoJefe(){
      return
dameInfoEmpleado()+"Departamento:"+departamento+"\n";
   }
};

int main()
{
   Empleado luis("Luis",31,18000);
   Jefe luisAlberto("Luis Alberto",32,30000,"Computacion");
   cout << "---\n" << luis.dameInfoEmpleado();
   cout << "---\n" << luisAlberto.dameInfoJefe();
   return 0;
}
```

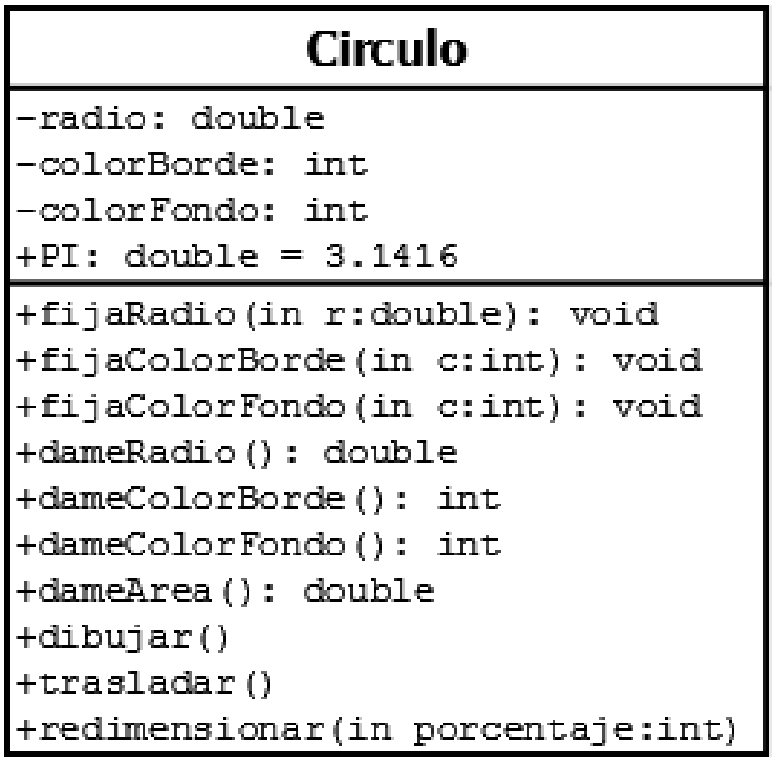
Véase ejemplo: “Ejemplos PRO\Orientada a Objetos\08 Herencia\03 Solucion”

La *herencia* es el mecanismo mediante el cual se aplica la reutilización de código en la programación orientada a objetos, esto nos permite abstraer en una clase *padre* todas las variables (*atributos*) y operaciones (*métodos*) que son comunes entre varias clases (*generalización*) y establecer en las clases hijas las demás variables y operaciones que no son comunes (*especialización*).

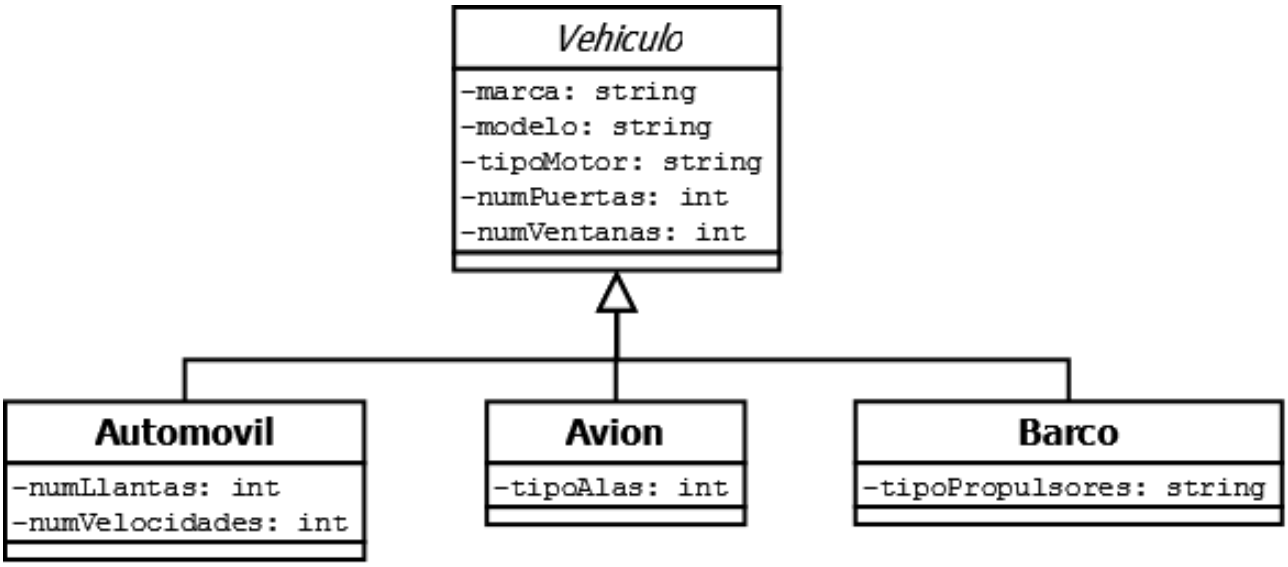
Reutilización de código mediante la herencia

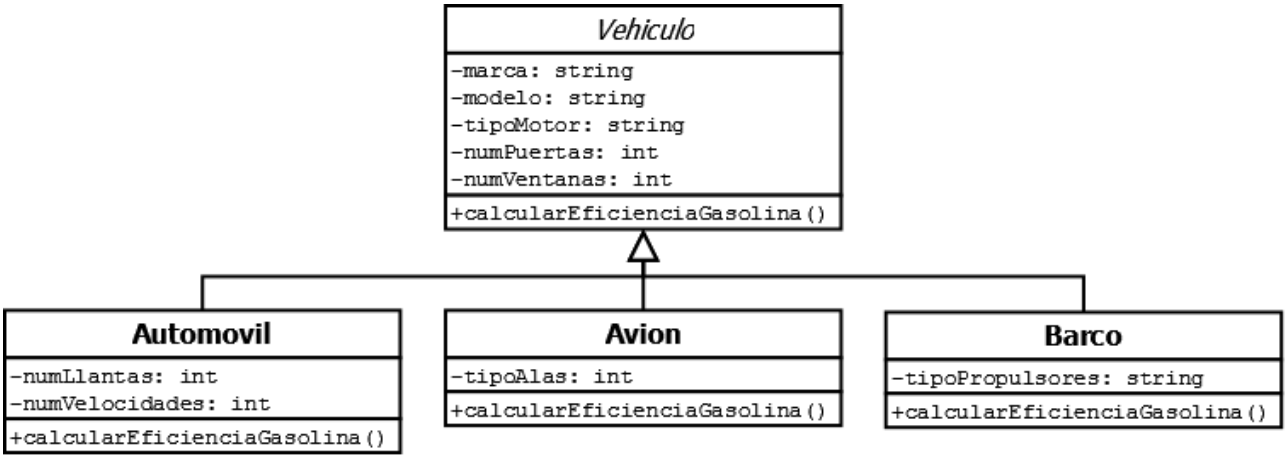


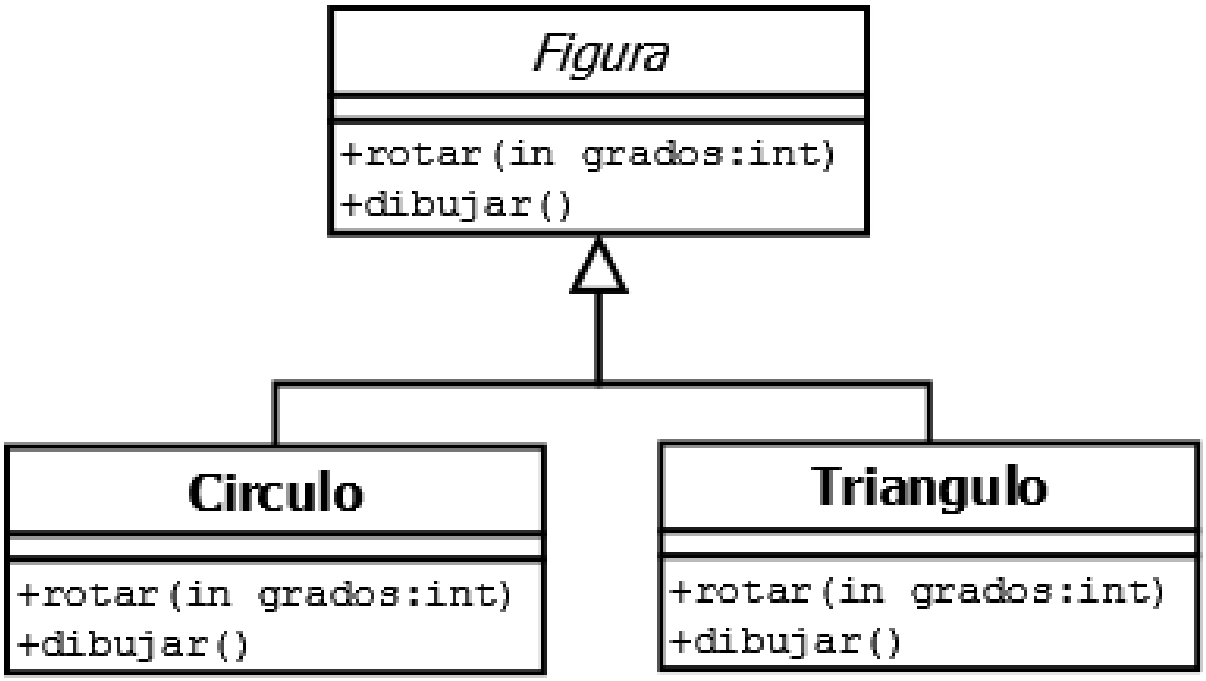
Métodos de Compotamiento



Clases Abstractas

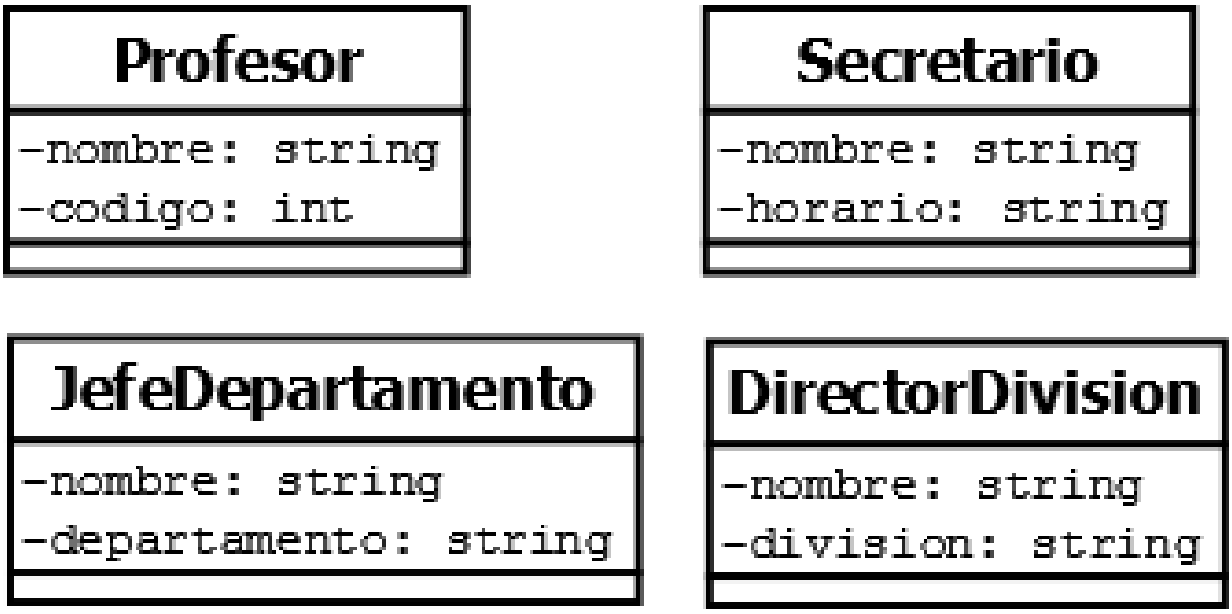




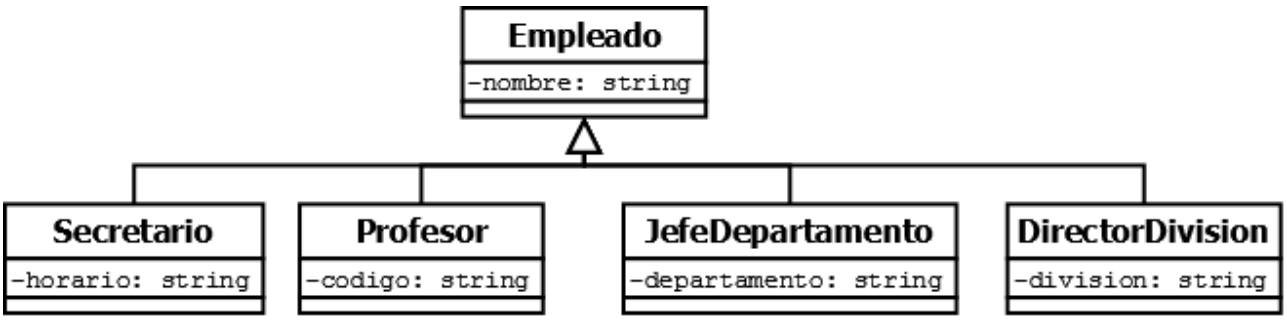


Generalización

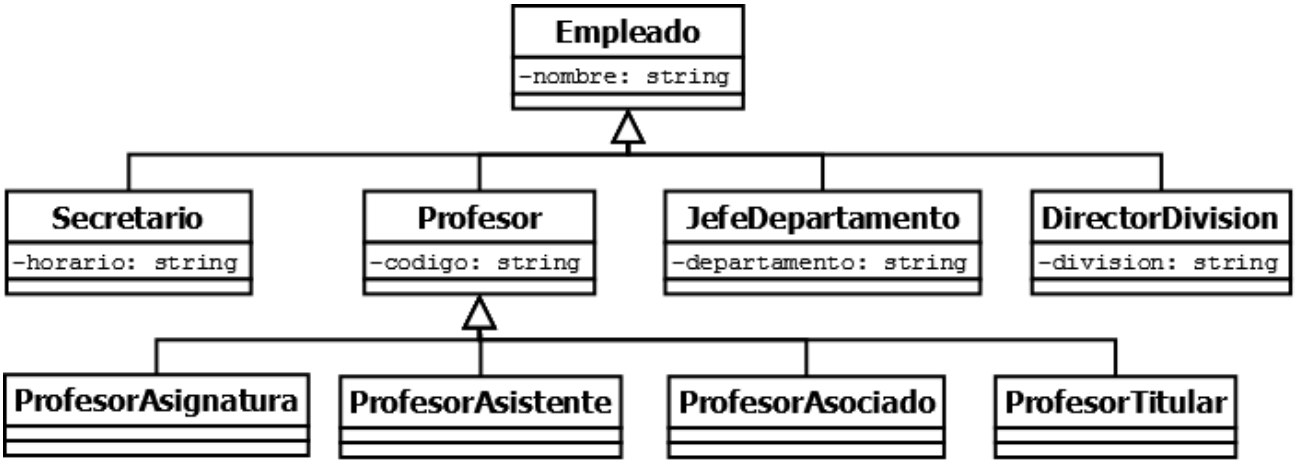
Sin Herencia:



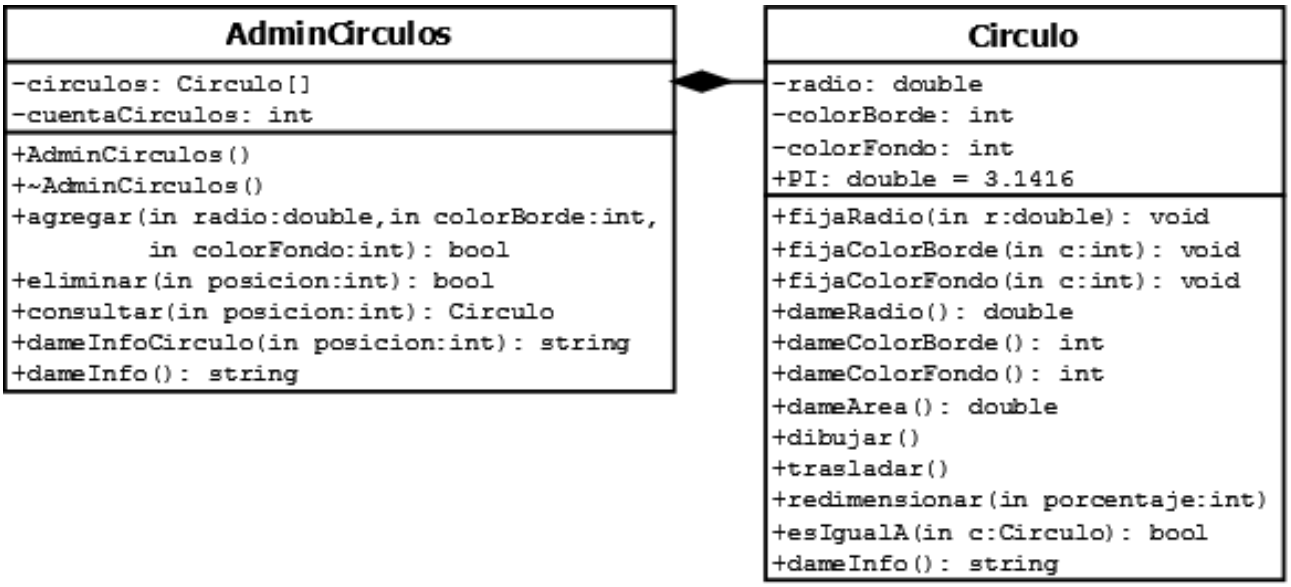
Con Herencia:



Especialización



Clase Administradora



Polimorfismo

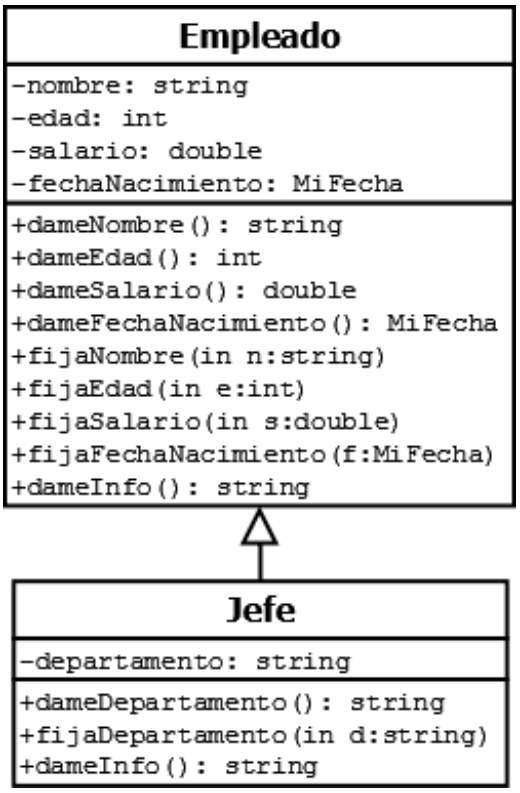
- Polimorfismo es la habilidad de tener muchas formas diferentes; por ejemplo la clase Jefe tiene acceso a los métodos de la clase Empleado
- Un objeto tiene únicamente una forma, si se instanció como Empleado es un Empleado, si se instanció como Jefe, es un Jefe
- El *polimorfismo* permite que un objeto se comporte acorde a la naturaleza de la información mediante la *sobrecarga* de métodos.

Véanse ejemplos en: “Ejemplos PRO\Orientada a Objetos\10 Polimorfismo\01 Sobrecarga”

- Una variable de instancia puede referenciar a objetos de diferentes formas
- Si una variable de instancia es de tipo Empleado, solo le son válidos de invocar los atributos y métodos (públicos) de la clase Empleado, aunque el objeto instanciado fuese por ejemplo de tipo Jefe

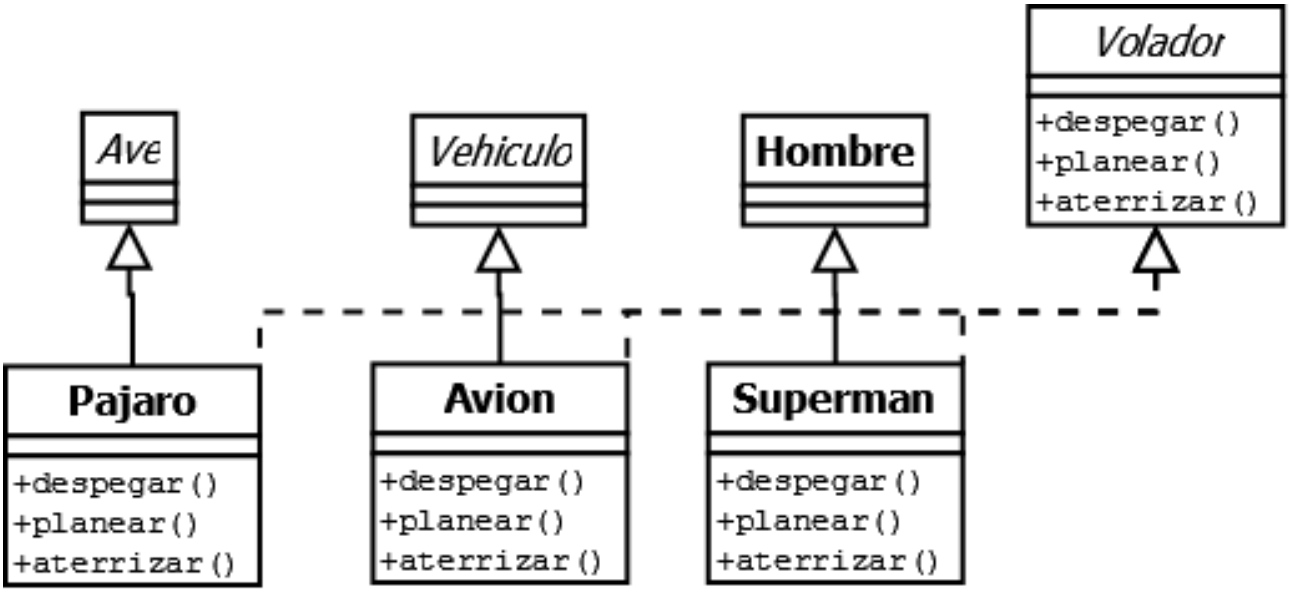
Véanse ejemplos en: “Ejemplos PRO\Orientada a Objetos\10 Polimorfismo\02 Variable de instancia”

- El *polimorfismo* permite que objetos de distintas clases se comporten acorde a la naturaleza propia (como clase) cuando contiene atributos y métodos heredados mediante la *sobreescritura* de métodos.



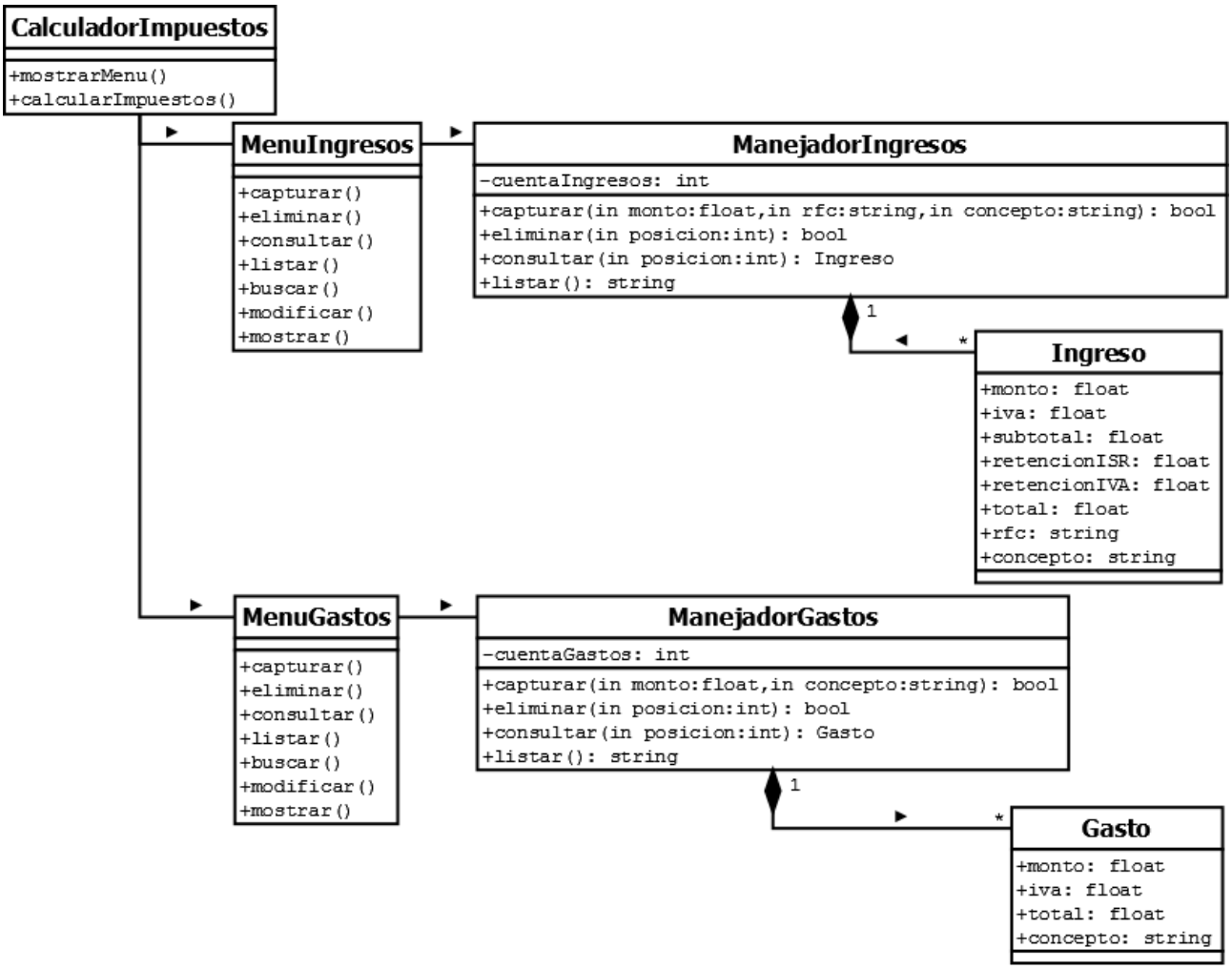
Véanse ejemplos en: “Ejemplos PRO\Orientada a Objetos\10 Polimorfismo\03 Sobreescritura y metodo virtual”

Herencia Múltiple - Interfaces



Véase ejemplo en: “Ejemplos PRO\Orientada a Objetos\10 Polimorfismo\04 Herencia multiple”

Diseño de Aplicaciones Orientado Objetos



- NOTA IMPORTANTE. Los objetos Ingreso y Gasto debieran tener sus atributos privados y sus métodos dameAtributo() (getters) y fijaAtributo() (setters que validen la información modelada)